Kafka Interview Questions

Contents

Section 1 - Architecture and Internals

Q1. Explain Kafka's overall architecture and the role of each core component.
Q2. What is a partition and why is it the fundamental unit of parallelism and ordering?
Q3. How does Kafka guarantee message ordering, and what can silently break it?
Q4. Explain the replication model: leader, follower, ISR, high watermark, and LEO.
Q5. What is the controller, and how does KRaft change it compared to ZooKeeper?
Q6. Why did Kafka move to KRaft, and what are the concrete operational benefits?
Q7. What is the high watermark, and how does it relate to durability and consumer visibility?
Q8. Explain leader election and unclean leader election. What are the trade-offs?
Q9. How does Kafka achieve such high throughput?
Q10. What is a log segment? How does Kafka find a record by offset?

Section 2 - Producers

Q11. Walk through the lifecycle of a produce request from send() to acknowledgement.
Q12. Explain acks=0, acks=1, and acks=all, and how min.insync.replicas interacts.
Q13. What is the idempotent producer and how does it work internally?
Q14. Explain producer batching. How do batch.size, linger.ms, and buffer.memory tune throughput vs latency?
Q15. How does the default partitioner work, including the sticky partitioner?
Q16. How does max.in.flight.requests.per.connection interact with retries, ordering, and idempotence?
Q17. Explain producer-side compression. Which codecs, and where does compression happen?

Section 3 - Consumers and Consumer Groups

Q18. Explain consumer groups and how partition assignment works.
Q19. Walk through the consumer poll loop. What actually happens during poll()?
Q20. Explain offset management: __consumer_offsets, auto vs manual commit, commitSync vs commitAsync.
Q21. Explain auto.offset.reset and exactly when it takes effect.
Q22. Explain consumer rebalancing. Contrast eager and cooperative incremental rebalancing.
Q23. Explain session.timeout.ms, heartbeat.interval.ms, and max.poll.interval.ms.
Q24. What is static group membership and what problem does it solve?
Q25. Compare partition assignment strategies: Range, RoundRobin, Sticky, CooperativeSticky.

Section 4 - Delivery Semantics and Reliability

Q26. Explain at-most-once, at-least-once, and exactly-once, and how to configure each.
Q27. Explain Kafka transactions and how exactly-once works in a consume-transform-produce loop.
Q28. What is the transaction coordinator and the __transaction_state topic?
Q29. What do read_committed and read_uncommitted mean, and when does it matter?
Q30. Enumerate the ways data loss can occur in Kafka and how to prevent it end to end.
Q31. How do duplicates occur, and how do you make a sink idempotent?

Section 5 - Storage, Retention, and Compaction

Q32. Explain retention: time-based vs size-based, and the cleanup.policy options.
Q33. Explain log compaction in detail, including tombstones and use cases.
Q34. When would you choose compaction vs deletion vs both?
Q35. What is tiered storage and why does it matter?

Section 6 - Ecosystem: Streams, Connect, Schema Registry

Q36. When do you use the Consumer API vs Kafka Streams vs Kafka Connect vs ksqlDB?
Q37. Explain Kafka Streams internals: state stores, changelog topics, and standby replicas.

Q38. Explain the difference between KStream, KTable, and GlobalKTable, and stream-table duality.
Q39. Explain Kafka Connect architecture: workers, connectors, tasks, converters, and SMTs.
Q40. Explain Schema Registry compatibility modes and a safe schema evolution strategy.

Section 7 - Scenario-Based Questions

Q41. Scenario: Consumer lag is steadily growing on one partition while others are fine. How do you diagnose and fix it?
Q42. Scenario: You must guarantee strict ordering per customer while sustaining very high throughput. How do you design it?
Q43. Scenario: You need exactly-once delivery from Kafka into a relational database. How do you achieve it?
Q44. Scenario: Your consumer group rebalances repeatedly, causing processing stalls. What is happening and how do you stop it?
Q45. Scenario: A downstream team accidentally deployed a new schema that breaks existing consumers. How do you prevent this class of incident?
Q46. Scenario: One broker is under heavy load and its partitions show high latency, while others are underutilized. How do you rebalance load?
Q47. Scenario: You need to reprocess the last 3 days of data through a fixed pipeline without disrupting live consumers. How do you do it?
Q48. Scenario: You must decide the partition count for a new high-volume topic. How do you reason about it?
Q49. Scenario: During an incident, acknowledged messages appear to have been lost. Walk through how you would root-cause it.
Q50. Scenario: A stateful Kafka Streams application is slow to recover after instance failures. How do you make failover fast and reliable?

Section 1 - Architecture and Internals

Q1. Explain Kafka's overall architecture and the role of each core component.

Kafka is a distributed, append-only commit log built for high-throughput, fault-tolerant, ordered event streaming. The core pieces are:

Broker - a single Kafka server that stores partition data, serves produce/fetch requests, and participates in replication. A cluster is a set of brokers.
Topic - a named logical stream of records. Topics are split into partitions.
Partition - an ordered, immutable, append-only log. It is the unit of parallelism, ordering, and replication. Each record within a partition gets a monotonically increasing offset.
Producer - client that writes records; it decides the target partition (by key hash, round-robin, or custom logic).
Consumer / Consumer Group - clients that read records. Partitions of a subscribed topic are distributed among consumers in a group so each partition is owned by exactly one consumer in that group.
Controller - one broker elected to manage cluster metadata: partition leadership, replica assignment, and broker membership.
Metadata layer - historically ZooKeeper; in modern Kafka this is KRaft, where a quorum of controllers stores metadata in an internal Raft log.

The key mental model is that Kafka is not a queue that deletes on read - it is a durable, replayable log. Consumers track their own position (offset), so many independent consumers can read the same data at their own pace, and retention is governed by time/size policy rather than by consumption.

Q2. What is a partition and why is it the fundamental unit of parallelism and ordering?

A partition is an ordered, append-only sequence of records identified by offsets. It is fundamental for three reasons:

Ordering - Kafka guarantees total order only within a partition, never across partitions of a topic. If two records must be processed in order, they must land in the same partition (typically by sharing a key).
Parallelism - within a consumer group, the maximum number of consumers that can work in parallel on a topic equals the number of partitions. Ten partitions cap useful consumers at ten; extra consumers sit idle.
Distribution and scalability - partitions are spread across brokers, so a topic's data and load are horizontally distributed. Adding partitions (and brokers) is how you scale a topic.

Because partitioning drives both ordering and throughput, choosing a partition key and partition count is one of the most consequential design decisions. Over-partitioning increases metadata, open file handles, end-to-end latency, and rebalance/recovery time; under-partitioning caps parallelism and can create hot partitions when keys are skewed.

Q3. How does Kafka guarantee message ordering, and what can silently break it?

Kafka guarantees per-partition ordering: records appended to a partition are read back in the same order. Same-key records hash to the same partition, so per-key ordering follows from co-partitioning. Ordering is NOT guaranteed across partitions.

Common ways ordering breaks:
1. Retries with in-flight parallelism. If the producer allows more than one in-flight request per connection and idempotence is disabled, a retried batch can be appended after a later batch that already succeeded, reordering records. Enabling the idempotent producer (default in modern Kafka) preserves order even with up to 5 in-flight requests because the broker enforces per-batch sequence numbers.
2. Changing partition count. Adding partitions changes the key-to-partition mapping (hash modulo partition count), so a key that used to go to partition 3 may now go to partition 7. Historical ordering per key is not preserved across that change.
3. Multiple producers to the same key. Ordering is only as strong as a single producer's send order; two producers writing the same key concurrently have no cross-producer ordering guarantee.
4. Consumer-side concurrency. If a consumer hands records from one partition to a thread pool, processing order is lost even though delivery order was correct.

Q4. Explain the replication model: leader, follower, ISR, high watermark, and LEO.

Each partition has one **leader** replica and zero or more **follower** replicas, placed on different brokers. All produce and consume traffic goes through the leader; followers passively fetch from the leader to stay current.

LEO (Log End Offset) - the offset of the next record to be written on a replica; effectively the tail of that replica's log.
ISR (In-Sync Replicas) - the set of replicas (including the leader) that are caught up with the leader within *replica.lag.time.max.ms*. A follower that stops fetching or falls behind is removed from the ISR and re-added once it catches up.
High Watermark (HW) - the highest offset that has been replicated to all members of the ISR. Only records below the high watermark are visible to consumers. This is what makes committed data durable: a consumer never sees a record until it exists on every in-sync replica, so a leader failure cannot expose then lose that record.

This design decouples durability (controlled by ISR + acks + min.insync.replicas) from availability (controlled by how the cluster reacts when the ISR shrinks). Understanding HW vs LEO is central to reasoning about data loss and consumer visibility.

Q5. What is the controller, and how does KRaft change it compared to ZooKeeper?

The controller is responsible for cluster-wide coordination: electing partition leaders, tracking broker liveness, propagating metadata changes, and handling replica reassignment.

ZooKeeper era: one broker was elected controller and used ZooKeeper as the source of truth for metadata (topics, partitions, ISR, configs). This had well known pain points - metadata scaled poorly to very large partition counts, controller failover required reloading state from ZooKeeper (slow), and operators had to run and tune a separate ZooKeeper ensemble.

KRaft (KIP-500): metadata is stored in an internal, replicated Raft log managed by a dedicated **controller quorum**. One controller is the active leader; others are hot standbys that already have the metadata log replicated. Benefits include much faster controller failover (no external reload), a single security and operational model, better scalability to millions of partitions, and no separate ZooKeeper deployment. ZooKeeper was deprecated and then removed in Kafka 4.0, so KRaft is now the standard. In interviews, being able to explain *why* the log-based metadata model scales better than the ZooKeeper-watch-based model is what distinguishes a senior answer.

Q6. Why did Kafka move to KRaft, and what are the concrete operational benefits?

The move addressed structural limits of the ZooKeeper design:

Scalability - metadata propagation via ZooKeeper watches did not scale to very high partition counts. KRaft models metadata as an ordered event log that brokers consume like any other log, so brokers apply metadata deltas incrementally instead of reloading full snapshots.
Faster failover - standby controllers already hold the replicated metadata log, so leadership transfer is near-instant rather than requiring a full reload from an external system.
Operational simplicity - one system to deploy, secure, monitor, and upgrade instead of Kafka plus ZooKeeper, each with its own configuration and failure modes.
Consistency model - a single Raft-based source of truth removes classes of split-brain and metadata-divergence issues.

Practically, this means quicker recovery during broker failures, support for far larger clusters, and simpler on-call operations. A nuanced point to raise: migrating an existing ZooKeeper cluster to KRaft is a staged process, and the controller quorum should be sized (typically 3 or 5 nodes) for the same availability reasons as any Raft/quorum system.

Q7. What is the high watermark, and how does it relate to durability and consumer visibility?

The high watermark (HW) is the highest offset replicated across all in-sync replicas of a partition. Its two jobs:

Consumer visibility - consumers can only read up to the high watermark. Records beyond the HW exist on the leader but are not yet acknowledged as durable across the ISR, so exposing them would risk showing data that could disappear on failover.
Durability boundary - because a record below the HW is present on every in-sync replica, the loss of the leader does not lose that record: a follower in the ISR (which also has it) can take over.

This ties directly into failure reasoning. With *acks=all* and *min.insync.replicas=2*, a produce request is acknowledged only after the record is on at least two in-sync replicas, which advances the HW safely. If the ISR shrinks below min.insync.replicas, the leader rejects writes (NotEnoughReplicas) rather than acknowledging data that is not sufficiently replicated. The gap between a replica's LEO and the HW is exactly the window of data that is written but not yet safely committed.

Q8. Explain leader election and unclean leader election. What are the trade-offs?

When a partition leader fails, the controller must pick a new leader. By default it chooses from the ISR, because those replicas are guaranteed to hold all committed (below-HW) records - this preserves durability with no data loss.

Unclean leader election occurs when no ISR replica is available (for example, all in-sync replicas are down) and *unclean.leader.election.enable=true*. Kafka then elects an out-of-sync replica as leader to restore availability. The trade-off is stark: the new leader may be missing records that were committed on the old leader, so those records are permanently lost, and its log truncation can cause other replicas to discard data to match.

The trade-off is availability vs durability (a CAP-style choice):
- *unclean.leader.election.enable=false* (default) - the partition becomes unavailable until an ISR replica returns, but no committed data is lost. This is correct for financial, audit, or source-of-truth data.
- *true* - the partition stays available at the cost of possible data loss, acceptable for high-volume metrics/telemetry where continuity matters more than a few lost records.

A strong answer notes that this must be reasoned about per topic, based on the cost of loss vs the cost of downtime.

Q9. How does Kafka achieve such high throughput?

Kafka's performance comes from working *with* the operating system rather than against it:

Sequential disk I/O - records are appended to the end of a log segment. Sequential writes on modern disks (including HDDs) are dramatically faster than random I/O, and reads are largely sequential too.
Page cache reliance - Kafka does not maintain a large in-process cache; it writes to the OS page cache and lets the kernel manage flushing and caching. Hot data is served straight from memory, and Kafka avoids duplicating data in the JVM heap (reducing GC pressure).
Zero-copy - on the consume path, Kafka uses sendfile to transfer bytes from the page cache directly to the network socket, bypassing user-space copies and CPU overhead.
Batching - producers group records into batches; consumers fetch in batches. Amortizing per-message overhead over large batches massively improves throughput.
Compression - producers compress whole batches (Snappy, LZ4, zstd, gzip), which reduces network and disk usage; Kafka can store and forward compressed batches without recompressing.
Binary protocol and no per-message ack - a compact protocol plus offset based positioning keeps the broker simple and stateless about consumer progress.

The unifying idea: minimize copies, prefer sequential access, and push batching end to end.

Q10. What is a log segment? How does Kafka find a record by offset?

A partition's log is not one giant file - it is split into **segments**, each a pair of files: the log file (the records) and index files. Segments roll over based on size (*segment.bytes*) or time (*segment.ms*). Only the active (newest) segment is written to; older segments are immutable and are the unit of retention and deletion.

Each segment is named by its base offset (the offset of its first record). Kafka maintains two sparse indexes per segment:
Offset index - maps a relative offset to a physical byte position in the log file, so Kafka can seek near a target offset without scanning.
Time index - maps timestamps to offsets, enabling offset-lookup-by-time (used by *auto.offset.reset* to a timestamp and by retention).

To read offset N, Kafka: (1) locates the segment whose base offset is the largest one not greater than N (a binary search over segment base offsets); (2) uses the offset index to jump to the nearest indexed position at or before N; (3) scans forward a short distance to the exact record. Because indexes are sparse, lookups are fast and memory-light. Segment structure also explains retention granularity: Kafka deletes whole segments, so a record can outlive its retention slightly until its segment is eligible for deletion.

Section 2 - Producers

Q11. Walk through the lifecycle of a produce request from send() to acknowledgement.

When you call *send()*:

1. Serialization - the key and value are serialized via configured serializers.
2. Partitioning - the partitioner picks a partition: by key hash if a key is present, otherwise via the sticky partitioner (batches to one partition until the batch is ready, then rotates) for efficiency.
3. Accumulation (buffering) - the record is appended to an in-memory batch in the RecordAccumulator, grouped per partition. This is where *batch.size*, *linger.ms*, and *buffer.memory* apply.
4. Sender thread - a background I/O thread drains ready batches and groups them by broker into produce requests, respecting *max.in.flight.requests.per.connection*.
5. Broker handling - the partition leader appends the batch to its log, followers replicate, and acknowledgement is governed by *acks* and *min.insync.replicas*.
6. Response / retries - on success the future/callback completes with metadata (partition, offset). On a retriable error (e.g. NotEnoughReplicas, leader change), the producer retries up to *retries* within *delivery.timeout.ms*. With idempotence enabled, retries do not create duplicates.

Because *send()* is asynchronous and buffered, throughput comes from the producer batching many records between network round trips, while durability comes from the broker-side acks/ISR settings.

Q12. Explain acks=0, acks=1, and acks=all, and how min.insync.replicas interacts.

acks controls when the producer considers a write acknowledged:

acks=0 - fire-and-forget. The producer does not wait for any broker acknowledgement. Highest

throughput, lowest latency, but records can be lost silently (e.g. leader down before write). Suitable only for lossy telemetry.
acks=1 - the leader acknowledges after writing to its own log, before followers replicate. If the leader fails after acking but before replication, that record is lost. A middle ground.
acks=all (or -1) - the leader acknowledges only after all in-sync replicas have the record. Combined with replication, this is the durable setting.

Crucially, *acks=all* alone is not enough. **min.insync.replicas** defines the minimum number of in-sync replicas required for a write to succeed. If the ISR shrinks below that number, the leader rejects the write with NotEnoughReplicas rather than acking under-replicated data. The classic durable configuration is replication.factor=3, min.insync.replicas=2, acks=all: you can lose one replica and still accept writes, and no acknowledged record is lost on a single broker failure. Setting min.insync.replicas=replication.factor removes tolerance for any replica being down (writes stop if one is unavailable), so 2 of 3 is the usual sweet spot.

Q13. What is the idempotent producer and how does it work internally?

The idempotent producer prevents duplicate records caused by producer retries. Without it, a transient failure where the broker actually persisted the batch but the ack was lost causes the producer to resend, creating a duplicate.

Internally, when *enable.idempotence=true* (the default in modern Kafka), each producer is assigned a **Producer ID (PID)** by the broker, and every record batch carries a monotonically increasing **sequence number** per partition. The broker tracks the last successfully written sequence number per (PID, partition). If it receives a batch with a sequence number it has already seen, it treats it as a duplicate and acknowledges without re-writing; if it sees a gap, it rejects with OutOfOrderSequence. This gives exactly-once *delivery into a single partition for the life of a producer session*.

Enabling idempotence also forces safe defaults: acks=all, retries greater than 0, and max.in.flight.requests.per.connection at most 5 (the broker can dedupe and reorder within that window while preserving order). Idempotence is per-producer-session and per-partition; it does not by itself provide cross-partition atomicity or exactly-once across a consume-process-produce pipeline - that requires transactions.

Q14. Explain producer batching. How do batch.size, linger.ms, and buffer.memory tune throughput vs latency?

The producer buffers records into per-partition batches before sending, which is the main throughput lever.

batch.size - the maximum size (bytes) of a single batch per partition. Larger batches improve throughput and compression ratio but a batch is not forced to fill before sending.
linger.ms - how long the producer waits to accumulate more records before sending a batch, even if it is not full. linger.ms=0 sends as soon as possible (low latency, smaller batches). Raising it (e.g. 5-50 ms) trades a little latency for much better batching and throughput under load.
buffer.memory - total memory for buffering unsent records. If it fills (producer outpacing the broker/network), *send()* blocks up to *max.block.ms*, then throws. This is your backpressure signal.

Tuning intuition:
- **Throughput-oriented:** larger batch.size, linger.ms of a few to tens of ms, enable compression (lz4/zstd), ample buffer.memory. Batches fill, compress well, and amortize network overhead.
- **Latency-oriented:** linger.ms near 0, modest batch.size, so records leave quickly.

The key insight: batch.size is a ceiling and linger.ms is a timer; the two together decide how big batches actually get, and compression multiplies the benefit of larger batches.

Q15. How does the default partitioner work, including the sticky partitioner?

The producer chooses a partition per record:

Keyed records - the partition is determined by a hash of the serialized key (murmur2) modulo the number of partitions. The same key always maps to the same partition (as long as partition count is unchanged), which is how per-key ordering is achieved.
Keyless records - older Kafka used round-robin per record, which produced many tiny batches. Modern Kafka uses the **sticky partitioner**: it sends all records to one partition until that partition's batch is full or ready, then switches to another partition. This dramatically improves batching and throughput while keeping load roughly even over time, since keyless records have no ordering requirement anyway.

Custom partitioners let you override this - for example, to route certain tenants to dedicated partitions, to implement weighted distribution, or to avoid a hot key by salting. A senior consideration: because keyed hashing is modulo partition count, increasing partitions remaps keys, so any system relying on per-key locality (co-partitioned joins, stateful processing) must be designed knowing that adding partitions changes key placement. If you need stable key placement across scaling, you plan partition count up front or use a custom scheme.

Q16. How does max.in.flight.requests.per.connection interact with retries, ordering, and idempotence?

This setting controls how many produce requests can be sent to a single broker connection without a response.

Without idempotence: if it is greater than 1 and retries are enabled, ordering can break. Suppose request A and request B are in flight; A fails and is retried while B has already succeeded - A now lands after B, reordering records in the partition. To strictly preserve order without idempotence, you must set it to 1, which lowers throughput.

With idempotence (enable.idempotence=true): the broker uses per-partition sequence numbers, so it can detect duplicates and enforce correct ordering even with multiple in-flight requests. Kafka therefore allows up to 5 in-flight requests while still guaranteeing both no duplicates and preserved order. This is why the idempotent producer is the recommended default - you keep high throughput (pipelined requests) and still get ordering and dedup.

Practical takeaway: enable idempotence and leave max.in.flight at its default (5). Only drop it to 1 in the rare case you cannot enable idempotence but still need strict order. Values above 5 are rejected when idempotence is on, because the broker's dedup window is bounded.

Q17. Explain producer-side compression. Which codecs, and where does compression happen?

Compression in Kafka is applied by the producer to an entire record *batch*, not per record, which is why larger batches compress far better. The codec is set via *compression.type*: none, gzip, snappy, lz4, or zstd.

Trade-offs:
gzip - best ratio, highest CPU cost.
snappy - fast, moderate ratio.
lz4 - very fast, good general default.
zstd - excellent ratio at competitive speed; often the best modern choice for network/storage savings.

Where work happens: the producer compresses the batch; the broker generally stores the batch still compressed and forwards it as-is, so it does not pay to recompress on the normal path (an exception is when broker-side re-compression is forced by a differing broker compression.type, which you usually avoid by keeping broker *compression.type=producer*). Consumers decompress on read. Because compressed batches travel and rest compressed, compression reduces network bandwidth, disk usage, and replication traffic simultaneously.

A subtle senior point: compression interacts with batching and latency. To get good ratios you want reasonably large batches (tune batch.size and linger.ms), so compression and batching should be tuned together, and zstd is commonly chosen when storage/egress cost dominates.

Section 3 - Consumers and Consumer Groups

Q18. Explain consumer groups and how partition assignment works.

A consumer group is a set of consumers that share a group.id and cooperatively consume a topic (or topics). Kafka assigns each partition of the subscribed topics to exactly one consumer within the group, so the group as a whole processes every partition once, in parallel.

Coordination is handled by a broker acting as the **group coordinator**. Consumers join the group, one is designated the leader, and the leader runs the configured assignment strategy to map partitions to members; the coordinator then distributes the assignment. Members send periodic heartbeats to prove liveness.

Key consequences:
- Parallelism is capped by partition count; extra consumers beyond the number of partitions stay idle.
- Different groups are independent - each group has its own offsets and reads all data, enabling fan-out

(e.g. an analytics group and a billing group both consume the same topic).
- Offsets are committed per (group, topic, partition) to the internal *__consumer_offsets* topic, so a group can resume where it left off after a restart.

When membership changes (a consumer joins, leaves, or is deemed dead), the group **rebalances** to redistribute partitions, which is a central operational concern discussed separately.

Q19. Walk through the consumer poll loop. What actually happens during poll()?

The consumer is driven by a loop calling *poll(timeout)*. A single *poll()* does far more than fetch records:

1. Coordinator / group management - it sends heartbeats (in modern clients heartbeats also run on a background thread), and if a rebalance is pending, poll() is where the consumer rejoins the group and receives its new assignment.
2. Fetching - it returns records already buffered from fetch requests and issues new fetch requests to partition leaders, honoring *fetch.min.bytes*, *fetch.max.wait.ms*, and *max.partition.fetch.bytes*.
3. Offset handling - if *enable.auto.commit=true*, offsets are committed periodically during poll() based on *auto.commit.interval.ms*.
4. max.poll.records - caps how many records a single poll() returns so your processing loop stays responsive.

The critical constraint is *max.poll.interval.ms*: the consumer must call poll() again within this window, or the coordinator assumes it is stuck, removes it from the group, and rebalances its partitions to others. So poll() must be called frequently - long, blocking per-record processing between polls is the classic cause of surprise rebalances. The healthy pattern is: poll a bounded batch, process it promptly (or offload safely while still polling), commit, and loop.

Q20. Explain offset management: __consumer_offsets, auto vs manual commit, commitSync vs commitAsync.

A consumer's **offset** is the position of the next record to read per partition. Committed offsets are stored in the internal compacted topic *__consumer_offsets*, keyed by (group, topic, partition), so a group resumes correctly after restart or rebalance.

Auto commit (*enable.auto.commit=true*) - offsets are committed periodically during poll(). Simple, but risky: it commits based on what was *polled*, not what was successfully *processed*, so a crash between poll and processing can lose records (at-most-once-ish) or, depending on timing, reprocess them. Adequate only when reprocessing/loss is tolerable.

Manual commit gives you control to commit only after processing succeeds:
- **commitSync()** - blocks until the broker confirms, retries on retriable errors, and surfaces failures. Safer, but adds latency and can throttle throughput.
- **commitAsync()** - fires the commit and continues, using a callback for the result. Higher throughput but no automatic retry (a later successful commit supersedes it anyway).

The common robust pattern: commitAsync() in the steady loop for throughput, plus a final commitSync() on

shutdown and after a rebalance to guarantee the last position is durably committed. For strict at-least-once, always process first, then commit.

Q21. Explain auto.offset.reset and exactly when it takes effect.

auto.offset.reset tells a consumer where to begin when there is **no valid committed offset** for its group on a partition - either a brand-new group or a committed offset that has aged out of retention and is no longer valid.

Values:
earliest - start at the beginning of the partition, reading all retained data. Good for reprocessing/backfill or when you must not skip data.
latest (default) - start at the end, reading only records produced after the consumer subscribes. Good for live streams where history is irrelevant, but a new group started with latest will silently skip all existing data.
none - throw an exception if no committed offset exists, forcing an explicit decision. Useful when starting without a known position is itself an error condition.

The critical nuance for interviews: this setting does *not* apply on every poll - only when there is no valid offset to resume from. It is also the recovery path if a committed offset points before the earliest available data (offset out of range), in which case the consumer resets per this policy. A frequent production bug is a new consumer group defaulting to latest and appearing to "miss data" that was actually produced before it started.

Q22. Explain consumer rebalancing. Contrast eager and cooperative incremental rebalancing.

A rebalance redistributes partitions among the members of a group. It is triggered when a consumer joins, leaves, or is considered dead (missed heartbeats or exceeded max.poll.interval.ms), when the subscription changes, or when partitions are added.

Eager rebalancing (classic): stop-the-world. Every consumer revokes *all* its partitions, then the whole group rejoins and receives a fresh assignment. During the revoke-to-reassign window, no one consumes - this is a throughput and latency hit, and it is worse in large groups or with frequent membership churn.

Cooperative incremental rebalancing (CooperativeStickyAssignor): only the partitions that actually need to move are revoked; consumers keep processing the partitions they retain. Rebalancing happens over one or more short rounds, avoiding a full pause. This is now the recommended approach and dramatically reduces disruption during scaling and deployments.

Complementary techniques to reduce rebalance pain:
- **Static membership** (group.instance.id) avoids rebalances on brief restarts by giving members a stable identity.
- Tuning session.timeout.ms / max.poll.interval.ms so healthy but slow consumers are not falsely evicted.

Senior candidates connect rebalances to real incidents: a slow downstream call inside the poll loop exceeding max.poll.interval.ms triggers repeated rebalances, creating a cascading stall.

Q23. Explain session.timeout.ms, heartbeat.interval.ms, and max.poll.interval.ms.

These three settings govern how the group detects a failed or stuck consumer, and they fail in different ways:

heartbeat.interval.ms - how often the consumer sends heartbeats to the coordinator to signal it is alive. Heartbeats run on a background thread, independent of record processing.
session.timeout.ms - if the coordinator receives no heartbeat within this window, it declares the consumer dead and rebalances its partitions. This catches process crashes, GC pauses, and network partitions. heartbeat.interval.ms is typically set to about one-third of session.timeout.ms so a couple of missed heartbeats do not immediately evict a healthy member.
max.poll.interval.ms - the maximum time allowed between successive poll() calls. This catches a different failure: a consumer that is alive (still heartbeating) but stuck processing a batch for too long. If exceeded, the consumer proactively leaves the group and a rebalance occurs.

The distinction matters: heartbeats prove the *process* is alive; max.poll.interval proves *progress* is being made. A common production symptom is a group that keeps rebalancing because per-batch processing occasionally exceeds max.poll.interval.ms - the fix is to lower max.poll.records, speed up processing, or increase the interval, not to touch heartbeats.

Q24. What is static group membership and what problem does it solve?

Static membership (KIP-345), enabled by assigning each consumer a stable *group.instance.id*, lets a consumer retain its identity - and therefore its partition assignment - across restarts.

The problem it solves: with dynamic membership, every time a consumer leaves and rejoins (a rolling deploy, a pod restart, a brief network blip), the group rebalances, pausing or shuffling work. In large groups or frequent-deploy environments, this churn is costly and can even cascade.

With a static group.instance.id, when that instance disconnects the coordinator does *not* immediately reassign its partitions; it waits for the same instance to return within session.timeout.ms. If the instance rejoins in time, it resumes its previous partitions with **no rebalance** at all. Only if it stays gone past the timeout does a rebalance occur.

This is especially valuable for stateful consumers (e.g. Kafka Streams instances with large local state), where a rebalance would otherwise force expensive state migration or restoration. The trade-off: if an instance truly dies, its partitions remain unassigned until the session timeout expires, so you tune the timeout to balance fast failure detection against tolerance for short restarts.

Q25. Compare partition assignment strategies: Range, RoundRobin, Sticky, CooperativeSticky.

The assignor (partition.assignment.strategy) decides how partitions map to consumers:

Range - assigns contiguous ranges of each topic's partitions per consumer. Simple, but with multiple topics it can skew load onto lower-indexed consumers, and co-partitioning across topics can be uneven.
RoundRobin - lays out all partitions across all consumers in round-robin order, giving more even distribution than Range, but on rebalance it can reshuffle many assignments unnecessarily.
Sticky - aims for balanced assignment while *minimizing movement* from the previous assignment during a rebalance, so most consumers keep most of their partitions. This reduces disruption and state churn.
CooperativeSticky - combines sticky balancing with the cooperative incremental rebalance protocol, so only the partitions that must move are revoked and the rest keep processing without a stop-the-world pause. This is the modern recommended default.

For stateful consumers or Kafka Streams, stickiness is important because moving a partition means rebuilding or migrating local state; CooperativeSticky both minimizes moves and avoids full pauses. A senior note: all members of a group must agree on the protocol, so migrating a running group to CooperativeSticky requires a careful rolling upgrade path.

Section 4 - Delivery Semantics and Reliability

Q26. Explain at-most-once, at-least-once, and exactly-once, and how to configure each.

These describe delivery/processing guarantees end to end:

At-most-once - records may be lost but never processed twice. Achieved by committing offsets *before* processing (or using auto-commit that runs ahead of processing). A crash after commit but before processing loses those records. Acceptable for lossy metrics.

At-least-once - records are never lost but may be processed more than once. Achieved by processing first and committing offsets *after* success, with durable producer settings (acks=all, retries). A crash after processing but before commit causes reprocessing on restart. This is the most common default; downstream logic must be idempotent to tolerate duplicates.

Exactly-once (EOS) - each record affects the result exactly once despite failures. In Kafka this is real for *Kafka-to-Kafka* pipelines using the idempotent producer plus transactions (transactional.id) so that produced records and consumed offsets commit atomically, with consumers reading *read_committed*. Kafka Streams exposes this via *processing.guarantee=exactly_once_v2*. For Kafka-to-external-system, true EOS requires either an idempotent sink (dedup by key) or a two-phase / transactional sink, because Kafka cannot make an arbitrary external write atomic with an offset commit.

The senior framing: exactly-once is achievable within Kafka's boundary; crossing into external systems reduces to idempotency or transactional integration.

Q27. Explain Kafka transactions and how exactly-once works in a consume-transform-produce loop.

Kafka transactions let a producer write to multiple partitions - and commit the consumer's input offsets - **atomically**, so a whole consume-transform-produce step either fully happens or fully does not.

Mechanics:
transactional.id - a stable identifier that ties a logical producer across restarts and enables zombie fencing. On init, the broker assigns a producer **epoch**; a newer instance with the same transactional.id fences out an older "zombie" instance so it can no longer commit.
Transaction coordinator - a broker-side component that tracks transaction state in the internal *__transaction_state* topic and drives the two-phase commit.
The loop - beginTransaction(); process records; produce output records; call *sendOffsetsToTransaction()* to include the consumed input offsets in the same transaction; then commitTransaction(). Because the offset commit and the output writes are in one atomic transaction, there is no window where output is written but offsets are not (duplicates) or offsets advance without output (loss).

On the read side, consumers set *isolation.level=read_committed* so they only see records from committed transactions, never aborted ones. This is exactly the model Kafka Streams uses under *exactly_once_v2*. The guarantee is confined to Kafka topics and offsets; it does not extend to arbitrary external side effects.

Q28. What is the transaction coordinator and the __transaction_state topic?

The transaction coordinator is a broker-side module (one per transactional producer, chosen by hashing transactional.id) that manages the lifecycle of transactions and enforces atomicity and fencing.

It persists transaction metadata in the internal, replicated, compacted topic **__transaction_state**: the current transaction's status (ongoing, prepare-commit, prepare-abort, complete), the set of partitions the transaction has written to, the producer id and epoch, and timeouts. Because this state is in a durable Kafka topic, the coordinator can recover after a broker failure and correctly finish or abort in-flight transactions.

On commit, the coordinator runs a two-phase protocol: it writes a prepare-commit marker, then writes **transaction markers** (commit/abort control records) into each data partition the transaction touched. These markers are how consumers with read_committed know which records became visible and which were aborted. The coordinator also enforces **fencing** via the producer epoch: if a newer producer instance registers the same transactional.id, the epoch is bumped and the older instance's commits are rejected, preventing a zombie from corrupting exactly-once semantics.

Understanding this explains why EOS adds some latency and internal topic overhead - it is a genuine distributed commit protocol layered on the log.

Q29. What do read_committed and read_uncommitted mean, and when does it matter?

isolation.level controls whether a consumer sees records that are part of open or aborted transactions:

read_uncommitted (default) - the consumer sees all records up to the high watermark, including records from transactions that are still open or that were ultimately aborted. Fine when you are not using

transactions, but it breaks exactly-once if you are, because you may consume records that get rolled back. **read_committed** - the consumer only sees records from *committed* transactions, plus all non-transactional records. Records from aborted transactions are filtered out, and records from an open transaction are held back until that transaction commits.

The important consequence: under read_committed, a consumer cannot advance past an in-flight transaction's records until it commits or aborts, because visibility is bounded by the **Last Stable Offset (LSO)** - the offset up to which all transactions are decided - rather than the high watermark. A long-running or stuck transaction therefore blocks read_committed consumers on that partition, which is why transaction timeouts and short transactions matter operationally.

In interviews, tie this to exactly-once: EOS output is only correct if downstream consumers read committed data; otherwise they would observe aborted writes and lose the guarantee.

Q30. Enumerate the ways data loss can occur in Kafka and how to prevent it end to end.

Data loss can enter at several layers; a robust design closes each:

Producer side: acks=0 or acks=1 can lose acknowledged-looking writes on leader failure. Fix: acks=all, retries greater than 0, enable idempotence, and set delivery.timeout.ms high enough that transient issues are retried rather than dropped.
Broker/replication side: if min.insync.replicas is too low (e.g. 1), a write can be acknowledged on a single replica and lost if that broker fails. Fix: replication.factor=3 with min.insync.replicas=2.
Unclean leader election: electing an out-of-sync replica discards committed records. Fix: keep unclean.leader.election.enable=false for durable topics.
Consumer side: committing offsets before processing (or auto-commit racing ahead) loses records on a crash. Fix: process-then-commit (at-least-once) and make downstream idempotent.
Retention: data can age out before a lagging consumer reads it. Fix: size retention for worst-case lag and alert on lag and on retention pressure.

End to end, the durable recipe is: acks=all + idempotence, RF=3 with min.insync=2, unclean election disabled, process-before-commit, generous retention, and monitoring of under-replicated partitions and consumer lag. The senior insight is that no single setting guarantees no loss - it is the combination across producer, broker, and consumer that does.

Q31. How do duplicates occur, and how do you make a sink idempotent?

Duplicates arise from the fundamentals of at-least-once delivery:

Producer retries - a lost ack causes a resend of an already-persisted batch (mitigated by the idempotent producer within a partition/session).
Consumer reprocessing - a crash or rebalance after processing but before committing offsets replays those records on restart.
Rebalances - a partition briefly owned by two consumers during a rebalance window can double-process boundary records.

Because at-least-once is the practical default for most pipelines, the durable strategy is to make the **sink idempotent** so duplicates are harmless:
- **Deterministic keys / upserts** - derive a stable business key and use INSERT ... ON CONFLICT / MERGE / upsert so replays overwrite rather than duplicate.
- **Dedup by unique id** - store a processed-id set (or unique constraint) keyed by something like (topic, partition, offset) or a business event id, and skip already-seen ids.
- **Idempotent operations** - design writes so applying them twice equals applying once (e.g. set a value rather than increment).
- **Transactional sink** - where the target supports it, commit the data and the Kafka offset together (or store the offset in the same transactional store) for true exactly-once into that system.

The framing to convey: you rarely eliminate duplicates on the wire; you design the destination to be indifferent to them.

Section 5 - Storage, Retention, and Compaction

Q32. Explain retention: time-based vs size-based, and the cleanup.policy options.

Retention decides how long data stays in a topic, independent of whether it has been consumed - this is what makes Kafka replayable.

Time-based - *retention.ms* keeps data for a duration (e.g. 7 days). Segments older than the threshold become eligible for deletion.
Size-based - *retention.bytes* caps the total size per partition; oldest segments are removed once the cap is exceeded.
When both are set, whichever triggers first applies. Deletion happens at **segment** granularity (the active segment is never deleted), so data can persist slightly beyond the nominal threshold until its segment rolls and becomes eligible.

cleanup.policy selects the mechanism:
- **delete** (default) - remove old segments per the retention limits above.
- **compact** - retain at least the latest value per key indefinitely (log compaction), independent of age.
- **compact,delete** - both: compact to keep the latest value per key, and also apply time/size retention to bound total growth.

Choosing correctly is a design decision: event streams that will be reprocessed use delete with retention sized for downstream lag and backfill needs; changelog/state topics that represent "current value per key" use compact so the latest state survives forever.

Q33. Explain log compaction in detail, including tombstones and use cases.

Log compaction guarantees that, for each key, Kafka retains **at least the most recent value**, pruning older values for the same key over time. Unlike time/size deletion, compaction is key-aware and can keep

the latest state indefinitely regardless of age.

How it works: a background **log cleaner** scans older (non-active) segments, builds a map of key to latest offset, and rewrites segments keeping only the newest record per key. The active segment (the head of the log) is not compacted, and there are tunables (min.cleanable.dirty.ratio, min.compaction.lag.ms, max.compaction.lag.ms) controlling when and how aggressively cleaning runs.

Tombstones - to delete a key, you produce a record with that key and a **null value**. Compaction treats it as a delete marker, removing prior values for the key; the tombstone itself is retained for at least *delete.retention.ms* so that all consumers have a chance to observe the deletion before it is finally purged.

Use cases: changelog/state topics (database CDC-style "latest row per primary key"), Kafka Streams state store changelogs, configuration or entity snapshots, and any "current value per key" store you want to be able to bootstrap by replaying the topic from the beginning. The mental model: a compacted topic behaves like a durable, replayable key-value snapshot plus a live update stream.

Q34. When would you choose compaction vs deletion vs both?

The choice follows from what the topic *represents*:

delete - use for event/fact streams where each record is an independent occurrence (clicks, transactions, logs, metrics). You care about a window of history, not "the latest per key," so you keep everything for a retention period sized to cover downstream lag, reprocessing, and backfill, then drop old data.

compact - use for state/changelog topics where each key has a current value and older values are obsolete (entity snapshots, CDC of a table keyed by primary key, Streams state changelogs, feature stores). Compaction lets you rebuild current state by replaying from offset 0 while bounding growth to roughly one record per key. Deletes are expressed as null-value tombstones.

compact,delete - use when you want the latest value per key *but also* must bound total retention by time or size - for example, a state topic where very old keys that are never updated again should eventually expire, or where you must cap disk. Compaction keeps the latest per active key; the delete policy still ages out data beyond the retention limit.

The senior framing: pick the policy by asking "is this a stream of events or a snapshot of state?" Events want delete; state wants compact; state that must not grow unbounded wants compact,delete.

Q35. What is tiered storage and why does it matter?

Tiered storage (KIP-405) separates Kafka's storage into two tiers: a **local tier** (fast broker disk holding recent data) and a **remote tier** (cheaper object storage such as S3/GCS/HDFS holding older data). Brokers offload older, closed log segments to remote storage while keeping recent segments locally, and consumers reading old data are served transparently from the remote tier.

Why it matters:
Cost and capacity - you can retain data for weeks or months (or effectively "forever") without provisioning

that much expensive local disk, because history lives in cheap object storage.
Elasticity and faster operations - because brokers hold less data locally, broker recovery, partition reassignment, and rebalancing move far less data, so scaling and failover are faster.
Decoupling retention from broker sizing - historically long retention forced large local disks; tiered storage breaks that coupling.

Trade-offs and caveats: reads from the remote tier have higher latency than local reads, so it is ideal for hot-recent + cold-historical access patterns; there are configs governing local retention vs total retention; and it shifts some cost to object-store request/egress. The strategic point for design discussions: tiered storage turns Kafka into a more viable long-term system of record and simplifies operating large clusters.

Section 6 - Ecosystem: Streams, Connect, Schema Registry

Q36. When do you use the Consumer API vs Kafka Streams vs Kafka Connect vs ksqlDB?

Each targets a different layer of a streaming system:

Consumer/Producer API - the low-level building block. Use it when you need full control, custom integration, or logic that does not fit a higher-level model. You manage threading, state, offsets, and fault tolerance yourself.

Kafka Streams - a JVM library for stateful stream processing *within your application* (no separate cluster). Use it for transformations, aggregations, windowing, joins, and enrichment where you want exactly-once, local state stores, and automatic scaling/rebalancing. Ideal for event-driven microservices doing real-time processing.

Kafka Connect - a framework for moving data *between Kafka and external systems* without writing code, via reusable source and sink connectors (databases, object stores, search, warehouses). Use it for ingestion and egress, CDC, and standard integrations; it handles scaling, offset tracking, and restart/recovery.

ksqlDB - a SQL layer over Kafka Streams for expressing stream processing declaratively. Use it when SQL is expressive enough and you want faster iteration without writing Java.

The decision heuristic: moving data in/out of Kafka -> Connect; processing data inside Kafka with code -> Streams; processing with SQL -> ksqlDB; bespoke or non-standard needs -> the raw client APIs.

Q37. Explain Kafka Streams internals: state stores, changelog topics, and standby replicas.

Kafka Streams provides stateful processing with fault tolerance by combining local state with Kafka-backed durability:

State stores - stateful operations (aggregations, joins, windows) keep local state, typically in embedded

RocksDB (on disk, spilling beyond memory) or in-memory stores. Local state makes lookups fast without hitting the network.
Changelog topics - every state store is backed by a compacted *changelog* topic in Kafka. Each update to local state is also written to the changelog, so the store's contents are durably captured. On failure or reassignment, Streams rebuilds the local store by replaying its changelog. Compaction keeps the changelog bounded to roughly the latest value per key.
Standby replicas (num.standby.replicas) - optional shadow copies of a state store maintained on other instances by consuming the changelog. If the active instance fails, a standby already has the state warm, so failover does not require a full changelog replay, sharply reducing recovery time.

Streams also uses **tasks** mapped to partitions for parallelism, supports **interactive queries** to read state stores directly, and offers exactly-once via processing.guarantee. The core idea to convey: local state gives speed, changelog topics give durability and rebuildability, and standbys give fast failover - the same log that stores data also stores the state derived from it.

Q38. Explain the difference between KStream, KTable, and GlobalKTable, and stream-table duality.

These are the core abstractions in Kafka Streams and map to two ways of interpreting a topic:

KStream - an unbounded *stream of independent events*. Every record is a new fact; two records with the same key are both meaningful (e.g. two purchases by the same user). Semantically it is insert-only / append.
KTable - a *changelog interpretation* of a stream, i.e. a materialized table where each key maps to its latest value. A new record for an existing key is an **update** (or a delete if the value is null / tombstone), not an additional event. A KTable is partitioned like the underlying topic, so each instance holds only its share of keys.
GlobalKTable - like a KTable but *fully replicated to every instance* rather than partitioned. Because every instance has the whole table, it enables joins on a key that is not the stream's partition key, without repartitioning - at the cost of storing a full copy on each instance. Ideal for small, relatively static reference/lookup data.

Stream-table duality is the unifying idea: a stream of updates can be folded into a table (the latest value per key), and a table's changes can be emitted as a stream of updates. This is why a KTable is backed by a compacted changelog and why you can convert between them. It also explains join semantics: KStream-KTable joins enrich each event with the current table value; KTable-KTable joins keep two materialized views consistent; GlobalKTable joins allow non-co-partitioned lookups. Articulating duality - "a table is a snapshot of a stream, a stream is the diff of a table" - is the mark of a senior answer.

Q39. Explain Kafka Connect architecture: workers, connectors, tasks, converters, and SMTs.

Kafka Connect is a distributed framework for reliably streaming data between Kafka and external systems:

Connectors - a source connector pulls from an external system into Kafka; a sink connector pushes from Kafka to an external system. A connector is configured, not coded (for standard systems).

Tasks - a connector is split into tasks for parallelism (e.g. one task per table or per partition set). Tasks are the unit of work distributed across workers.
Workers - the JVM processes that run tasks. In **distributed mode**, multiple workers form a cluster that shares load and rebalances tasks on failure, storing connector configs, offsets, and status in internal Kafka topics. Standalone mode runs a single worker (dev/simple use).
Converters - handle serialization between Kafka's byte records and the connector's data model (e.g. AvroConverter with Schema Registry, JSON, Protobuf). Converters are where schema handling lives.
SMTs (Single Message Transforms) - lightweight, per-record transformations applied in the pipeline (rename fields, mask data, route topics, add metadata) without a full stream-processing job.

Connect handles offset tracking and restart/recovery: source offsets and sink consumer offsets are managed so a restart resumes correctly. The value proposition to express: Connect standardizes and operationalizes integration - scaling, fault tolerance, and offset management come for free instead of being reimplemented per pipeline.

Q40. Explain Schema Registry compatibility modes and a safe schema evolution strategy.

Schema Registry stores and versions schemas (Avro, Protobuf, JSON Schema) and enforces **compatibility** rules so producers and consumers can evolve independently without breaking each other. It also lets messages carry a small schema id instead of the full schema, saving space.

Compatibility modes govern what changes are allowed for the next version:
BACKWARD (common default) - new schema can read data written with the previous schema; consumers upgrade first. Safe changes: add optional fields (with defaults), remove fields.
FORWARD - old schema can read data written with the new schema; producers upgrade first. Safe changes: add fields, remove optional fields.
FULL - both backward and forward compatible; the most restrictive/safe.
NONE - no checks (dangerous).
TRANSITIVE variants (BACKWARD_TRANSITIVE, etc.) check compatibility against *all* prior versions, not just the last one.

A safe evolution strategy: pick BACKWARD (or FULL for strong guarantees) as the default; always add fields with defaults and avoid renaming or changing types in place; treat field removal carefully per mode; and, for ordering of deploys, remember BACKWARD implies upgrade consumers first while FORWARD implies upgrade producers first. The senior point: compatibility mode encodes your deployment choreography, so choose it based on who you can upgrade first and enforce it in CI so breaking schemas are rejected before they reach production.

Section 7 - Scenario-Based Questions

Q41. Scenario: Consumer lag is steadily growing on one partition while others are fine. How do you diagnose and fix it?

Lag isolated to a single partition points to skew, not overall under-capacity.

Diagnose:
- Check whether that partition has a **hot key**. If most traffic shares one key (e.g. a whale customer id), all its records land on one partition, overloading the single consumer that owns it. Inspect key distribution and per-partition throughput.
- Check for a **slow or failing record** on that partition - a poison message that repeatedly errors and stalls progress, or a downstream dependency (a specific DB shard, an external API) that is slow only for that key range.
- Check consumer health for that partition's owner: GC pauses, resource limits, or uneven processing cost per record.

Fix:
- If it is a hot key, rework the partitioning: add a salt/sub-key to spread the hot entity across partitions (accepting loss of strict per-entity ordering if acceptable), or route that entity to a dedicated topic/partition with more resources.
- If it is a poison record, add error handling / dead-letter-queue routing so one bad record does not block the partition.
- If it is a downstream bottleneck, parallelize or cache that dependency, or apply backpressure.

The key insight to state: adding consumers will not help a single hot partition because one partition is owned by exactly one consumer in the group - you must fix the skew at the partitioning or processing layer.

Q42. Scenario: You must guarantee strict ordering per customer while sustaining very high throughput. How do you design it?

The tension is that ordering forces serialization per key while throughput wants parallelism. Resolve it by scoping ordering to the customer, not the whole topic:

Partition by customer id. Use the customer id as the record key so all of a customer's events map to the same partition, giving per-customer total order. Different customers land on different partitions and are processed in parallel, so aggregate throughput scales with partition count.

Choose partition count for parallelism and headroom. Enough partitions to spread customers evenly and to allow future consumer scaling, but not so many that metadata/latency suffers. Watch for skew: a few very large customers can create hot partitions - mitigate with a composite key (customer id plus a bounded sub-key) only if the business allows sub-ordering, or by isolating whales.

Preserve order on the producer. Enable idempotence (so retries do not reorder) and keep a single logical producer per key path; do not fan a customer's events across multiple producers.

Preserve order on the consumer. Process each partition on a single thread, or if you offload work, key-affinitize so a given customer's records are handled sequentially. Avoid a shared thread pool that reorders within a partition.

The crisp summary: ordering is a per-key property delivered by co-partitioning; throughput comes from

having many keys across many partitions processed in parallel.

Q43. Scenario: You need exactly-once delivery from Kafka into a relational database. How do you achieve it?

Kafka's native exactly-once covers Kafka-to-Kafka; writing to an external DB requires bridging the offset commit and the DB write. There are two solid patterns:

1. Idempotent upserts (most common, simplest). Run at-least-once (process-then-commit), but make the DB write idempotent: derive a stable key and use INSERT ... ON CONFLICT / MERGE so replays overwrite instead of duplicating, or store a unique event id / (topic, partition, offset) with a unique constraint and skip already-seen ids. Duplicates on the wire become harmless because the DB converges to the same state.

2. Store the offset in the same DB transaction (transactional outbox/inbox). In one database transaction, write the business data *and* the consumed offset into an offsets table; on restart, seek the consumer to the offset stored in the DB rather than trusting Kafka's committed offset. Because the data and the offset commit succeed or fail together atomically in the DB, you get true exactly-once into that system.

Notes: Kafka Connect sink connectors typically implement pattern 1 (idempotent upserts keyed appropriately). Avoid the naive approach of "write to DB, then commit to Kafka" as two independent steps - a crash between them yields duplicates or loss. The principle to articulate: since you cannot make an arbitrary external write atomic with a Kafka offset commit, you either make the sink idempotent or move the offset into the sink's own transaction.

Q44. Scenario: Your consumer group rebalances repeatedly, causing processing stalls. What is happening and how do you stop it?

Repeated rebalances almost always mean members keep leaving and rejoining the group. Work through the causes:

Processing exceeds max.poll.interval.ms. The most common cause: a batch takes too long to process, the consumer misses its poll deadline, the coordinator evicts it, partitions reassign, and the cycle repeats. Fix by lowering *max.poll.records* so batches are smaller, speeding up per-record work, offloading slow work while continuing to poll, or raising max.poll.interval.ms.

Heartbeat/session issues. Long GC pauses, undersized session.timeout.ms, or network blips cause missed heartbeats and eviction. Fix by tuning session.timeout.ms and heartbeat.interval.ms and addressing GC/resource pressure.

Deployment churn. Rolling restarts or autoscaling cause members to leave and rejoin, each triggering a rebalance. Fix with **static membership** (group.instance.id) so brief restarts do not rebalance, and use the **CooperativeSticky** assignor so rebalances that do happen are incremental rather than stop-the-world.

Diagnosis approach: inspect consumer logs for "leaving group" / "revoking partitions" messages,

correlate with processing time and GC logs, and check whether it aligns with deploys. The senior framing: distinguish "process is dead" (heartbeat / session timeout) from "process is stuck" (max.poll.interval) - the two have different fixes, and reaching for the wrong knob is a common mistake.

Q45. Scenario: A downstream team accidentally deployed a new schema that breaks existing consumers. How do you prevent this class of incident?

This is a schema-governance failure, and the fix is process plus tooling, not a one-off patch.

Immediate response: identify the offending schema version, roll the producer back to the last compatible schema, and if bad records were written, decide whether consumers can skip/dead-letter them or whether a corrective republish is needed.

Structural prevention:
- **Enforce Schema Registry compatibility.** Set a compatibility mode (BACKWARD or FULL, TRANSITIVE if you must stay compatible with all prior versions) so the registry *rejects* incompatible schemas at registration time rather than at runtime.
- **Shift validation left into CI.** Validate schema changes against the registry in the build pipeline, so a breaking change fails the merge/deploy, never reaching production.
- **Codify evolution rules:** add fields with defaults, avoid renames and in-place type changes, and treat removals per the compatibility mode. Deploy order matters - BACKWARD implies consumers upgrade first, FORWARD implies producers first.
- **Contract ownership:** treat the schema as a shared contract with clear ownership and review, and version topics or use subject naming strategies to isolate blast radius.

The message to convey: incompatible schemas should be impossible to deploy, not merely discouraged - compatibility enforcement in the registry plus CI turns a runtime outage into a caught build failure.

Q46. Scenario: One broker is under heavy load and its partitions show high latency, while others are underutilized. How do you rebalance load?

Uneven broker load usually means uneven partition/leadership placement or a few hot partitions concentrated on one broker.

Diagnose first:
- Check **leader distribution**: even if replicas are balanced, leadership can be skewed, and all produce/consume traffic hits leaders. Preferred-leader imbalance concentrates load.
- Check for **hot partitions** (skewed keys) living on that broker.
- Check disk, network, and under-replicated partitions on the hot broker.

Remediate:
- Run **preferred leader election** to redistribute leadership back to the intended balanced assignment.
- Use the **partition reassignment** tooling (kafka-reassign-partitions) to move replicas/leaders off the hot broker onto underutilized ones; throttle the reassignment so replication traffic does not itself overload the cluster.
- If a specific partition is hot due to a key, address the skew at the producer (salting/repartitioning) rather

than only moving it.
- For ongoing balance, consider automated balancing tooling and enable rack awareness so replicas are spread across failure domains.

Prevent recurrence: monitor per-broker throughput, leader counts, under-replicated partitions, and request latency, and alert on imbalance. The senior insight: moving partitions treats the symptom; if the root cause is key skew, you must fix partitioning, and if it is leadership skew, preferred-leader election is the right, lightweight fix.

Q47. Scenario: You need to reprocess the last 3 days of data through a fixed pipeline without disrupting live consumers. How do you do it?

The goal is a historical replay that does not interfere with the production consumer group's offsets or throughput.

Use a separate consumer group. Because offsets are per group, start a new group.id for the reprocessing job. It has independent offsets, so resetting its position does not touch the live group.

Seek to the right start point. Since you want the last 3 days, use offset-for-time to translate a timestamp (now minus 3 days) into starting offsets per partition (the time index supports this), or reset the new group to those offsets with the consumer group reset tooling. Ensure retention actually covers 3 days; if not, the data may already be gone (or in tiered storage, if enabled).

Isolate resources. Run the replay on its own consumers so it does not steal the live group's partitions. Watch cluster load - a large replay adds fetch traffic; use throttling or off-peak scheduling if needed, and be mindful of read amplification if old segments are in the remote (tiered) tier.

Make it idempotent. If the reprocessed output goes to the same sink, ensure upsert/dedup semantics so replayed records converge rather than duplicate.

The clean summary: reprocessing in Kafka is "spin up an independent consumer group and seek it backward (by time or offset)," which is safe precisely because consumers track their own position and the log is retained and replayable.

Q48. Scenario: You must decide the partition count for a new high-volume topic. How do you reason about it?

Partition count is a capacity and parallelism decision with long-lived consequences, so reason from both throughput and operational limits.

Throughput sizing. Estimate target throughput and the per-partition throughput your producers and (more often the binding constraint) your consumers can sustain. Roughly, partitions >= max(target/producer-per-partition, target/consumer-per-partition). Consumer processing cost usually dominates: if one consumer handles X records/sec and you need N times that, you need at least N partitions to allow N parallel consumers.

Parallelism ceiling. Remember the number of partitions caps the number of useful consumers in a group, so leave headroom for future growth in consumer count.

Costs of too many. Each partition adds open file handles, memory, more replicas to manage, longer leader-election and recovery times, higher end-to-end latency, and larger rebalance/reassignment cost. Very high partition counts also stress the controller (less so under KRaft, but still real).

Reversibility. You can add partitions later, but that remaps keyed data and can break per-key ordering and co-partitioned joins; you cannot easily reduce partitions. So slightly over-provision, but do not wildly over-partition.

The senior conclusion: pick partitions from expected peak throughput and desired consumer parallelism with modest headroom, account for skew, and treat the count as hard to shrink - so choose deliberately rather than defaulting.

Q49. Scenario: During an incident, acknowledged messages appear to have been lost. Walk through how you would root-cause it.

Treat "acknowledged but lost" as a durability-configuration or failover problem and work the layers systematically.

1. Confirm the producer's durability settings. Was *acks* set to 0 or 1? With acks=1, a leader can ack then fail before followers replicate, losing the record. Check acks, whether idempotence was enabled, and delivery.timeout.ms/retries.

2. Check replication settings. What are replication.factor and min.insync.replicas? If min.insync.replicas was 1, a write could be acked on a single replica and lost when that broker failed. Verify the ISR history around the incident.

3. Look for unclean leader election. If unclean.leader.election.enable was true and the ISR emptied, an out-of-sync replica could have become leader and truncated committed records. Check controller logs for unclean elections during the window.

4. Distinguish loss from consumer-side skips. Confirm the data truly is absent from the partition (not just skipped by a consumer that committed offsets before processing, or a new group using auto.offset.reset=latest). Inspect the log directly at the relevant offsets.

5. Check retention. Could the data have aged out before consumption due to short retention plus lag?

Remediation: move to acks=all, RF=3, min.insync=2, idempotence on, and unclean election off; add monitoring for under-replicated partitions and ISR shrink. The disciplined framing: loss of acknowledged data means either the ack was weak (acks/min ISR) or a failover discarded committed data (unclean election) - and you prove which by reading logs and metrics from the incident window, not by guessing.

Q50. Scenario: A stateful Kafka Streams application is slow to recover after instance failures. How do you make failover fast and reliable?

Slow recovery in Streams almost always means large local state is being rebuilt from changelog topics on failover. Attack it on several fronts.

Standby replicas. Set num.standby.replicas to 1 (or more). Standbys continuously consume the state stores' changelog on other instances, keeping a warm copy. On failure, a standby takes over with state already near-current, avoiding a full changelog replay - this is the single biggest recovery-time win.

Static membership + cooperative rebalancing. Give instances a stable group.instance.id so brief restarts (deploys, pod bounces) do not trigger reassignment and state migration at all, and use cooperative rebalancing so unaffected tasks keep running.

Keep changelogs healthy. Ensure state store changelog topics are compacted and appropriately sized so replay, when needed, is as small as possible; avoid unbounded state growth.

Right-size state and instances. Tune RocksDB memory, keep partitions/tasks balanced, and ensure disks are fast, since state restoration is I/O bound. Balance task/partition assignment so no single instance holds disproportionate state.

Faster failure detection tuning. Balance session.timeout so genuine failures are detected promptly without over-eager eviction.

The senior summary: recovery time is dominated by how much state must be rebuilt, so the strategy is "don't rebuild from scratch" - warm standbys plus stable membership convert a long changelog replay into a quick handover, and cooperative rebalancing limits the blast radius of any single failure.
